Lingroove — Project Guide

Version: MVP (as implemented in repository)

Purpose: This document explains what Lingroove is, what technologies and libraries it uses, how the code is organized, how data flows through the system, and what is intentionally simplified for an MVP versus what is planned for later.

---

1. Product summary

Lingroove is a web application that helps users learn Spanish through music by:

Accepting lyrics either as a URL (fetched and turned into plain text) or raw pasted text.

Cleaning the lyrics text for analysis.

Running NLP (spaCy) to extract vocabulary and classify tokens as verbs, nouns, or adjectives.

Storing songs, lyrics, and vocabulary entries in PostgreSQL.

Supporting playlists (create + fetch with song list and vocabulary count).

Exporting selected vocabulary rows to an Anki-compatible CSV with fixed column headers.

The frontend is a dark, music-app-inspired UI built with Next.js and Tailwind CSS. The backend is FastAPI with a modular service layer so translation and NLP can be swapped or extended later (e.g. real translation APIs, richer lyric parsers, Spotify metadata).

---

2. Repository layout (monorepo)

Path	Role
`apps/web/`	Next.js (App Router) frontend
`apps/api/`	FastAPI backend, SQLAlchemy models, Alembic migrations, services
`infra/`	`docker-compose.yml` — local PostgreSQL
`.env` / `.env.example`	Shared environment variable template (API URL, DB URL, CORS, spaCy model name)
`docs/`	This guide (source for the PDF)
`README.md`	Setup, first-time install, and day-to-day startup

Why a monorepo: Keeps frontend and backend in one place while remaining deployable separately (different processes, different dependencies). Matches the requirement for clean separation without forcing a multi-repo workflow during MVP development.

---

3. Technology stack

3.1 Frontend

Technology	Role
Next.js 15 (App Router)	Routing, server/client components, production build
React 19	UI rendering
TypeScript	Type safety for components and API types
Tailwind CSS	Utility-first styling; theme tokens for dark “Spotify-like” look
PostCSS + Autoprefixer	Tailwind pipeline
ESLint + eslint-config-next	Linting

Why Next.js: Required by the spec; App Router fits page-based MVP (landing, dashboard, analysis, playlist, export).

Why Tailwind: Fast iteration on a consistent dark UI without maintaining large custom CSS files.

3.2 Backend

Technology	Role
Python 3.11+ (project declares `>=3.11`; local may be newer)	Runtime
FastAPI	HTTP API, automatic request/response validation via Pydantic models
Uvicorn	ASGI server for local/production serving
SQLAlchemy 2.x	ORM: models, sessions, queries
psycopg (v3)	PostgreSQL driver (`postgresql+psycopg://` in `DATABASE_URL`)
Alembic	Database migrations
Pydantic Settings	Loads `.env` into a typed `Settings` object
spaCy	Spanish NLP: tokenization, POS tags, lemmas
requests	HTTP GET for lyric URLs
BeautifulSoup4	Parse HTML from URLs into plain text

Why FastAPI: Async-friendly, excellent typing and OpenAPI docs, fits a growing API surface.

Why SQLAlchemy + Alembic: Standard, migration-safe persistence for relational data (users, songs, vocab, playlists).

Why spaCy: Required by spec; provides deterministic POS/lemma pipeline without calling external LLMs in MVP.

3.3 Data store and infrastructure

Technology	Role
PostgreSQL 16 (Docker)	Primary database
Docker Compose	One-command local Postgres with persisted volume

Why PostgreSQL: Required by spec; strong fit for relational data and future features (progress, sync, etc.).

---

4. Installed dependencies (by package)

4.1 Backend — apps/api/pyproject.toml

Runtime

- fastapi — Web framework and routing.
- uvicorn[standard] — Dev/prod ASGI server (includes useful extras).
- sqlalchemy — ORM and query builder.
- psycopg[binary] — Postgres connectivity.
- alembic — Schema versioning and upgrades.
- pydantic-settings — Environment configuration.
- python-multipart — Prepared for form uploads (MVP primarily uses JSON).
- spacy — NLP pipeline.
- requests — Fetch remote lyric pages.
- beautifulsoup4 — HTML → text extraction.

Development

- pytest — Tests.
- httpx — HTTP client often used with Starlette/FastAPI test clients.

Additional manual step (not in pip alone): Spanish model download:

```
python -m spacy download es_core_news_md
```

If the model is missing, the code falls back to spacy.blank("es"), which has weaker POS/lemmatization than the full model.

4.2 Frontend — apps/web/package.json

- next, react, react-dom — Application framework.
- typescript, @types/* — Types.
- tailwindcss, postcss, autoprefixer — Styling pipeline.
- eslint, eslint-config-next — Lint rules.

Installed locally via: npm install inside apps/web/ (creates node_modules and package-lock.json).

---

5. Configuration and environment variables

Defined in .env.example and loaded by backend (pydantic-settings) and frontend build/runtime (NEXT_PUBLIC_*).

Variable	Purpose
`DATABASE_URL`	SQLAlchemy URL for Postgres (e.g. `postgresql+psycopg://user:pass@host:5432/db`)
`API_HOST` / `API_PORT`	Documented defaults for running Uvicorn (actual command may pass `--port` explicitly)
`CORS_ORIGINS`	Comma-separated browser origins allowed to call the API (e.g. `http://localhost:3000`)
`SPACY_MODEL`	Name of spaCy model to load (default `es_core_news_md`)
`TRANSLATION_PROVIDER`	Reserved for future real translation backends; MVP uses a small stub
`NEXT_PUBLIC_API_BASE_URL`	Base URL for frontend `fetch()` calls (must include `/api/v1`)
`APP_ENV`	Environment label (e.g. `development`)

Why NEXT_PUBLIC_*: Next.js inlines these for browser code so the client knows where the API lives.

---

6. Database schema (logical model)

Tables (Alembic migration 0001_initial + SQLAlchemy models in app/models/models.py):

users

- id, email (unique), display_name, created_at
- MVP note: The UI sends userId: 1. On first import, if that user does not exist, import-lyrics auto-creates a user with that id and a placeholder email (user{id}@lingroove.local). This avoids foreign-key errors before real authentication exists.

songs

- Belongs to users (user_id).
- title, artist, source_type (url | raw), source_url (when URL), external_track_id (reserved for future Spotify track id — not used in MVP logic).

lyrics

- Belongs to songs.
- raw_text, clean_text, language (currently defaulted to es), created_at.

vocabulary_entries

- Belongs to song and lyric.
- original_word, lemma, infinitive_form (for verbs), english_translation, context_line, part_of_speech (verb | noun | adjective), is_selected, created_at.

- Index on (song_id, part_of_speech) for faster grouping/filtering.

playlists

- Belongs to users.
- name, description, created_at.

playlist_songs

- Join table: playlist_id, song_id, added_at.
- Unique constraint on (playlist_id, song_id) so the same song is not duplicated in a playlist.

Cascade deletes: Deleting a parent (e.g. user or song) removes dependent rows according to FK ondelete rules defined in models/migration.

---

7. Backend application structure

7.1 Entry point — app/main.py

- Creates FastAPI app.
- Adds CORS middleware using settings.cors_origins (split by comma).
- Mounts routers under prefix /api/v1:
- lyrics, analysis, anki, playlists
- Exposes GET /health for simple uptime checks.

Note: Table creation is not auto-run on startup; schema is expected to come from Alembic (alembic upgrade head).

7.2 Core — app/core/

- config.py — Settings singleton (database_url, cors_origins, spacy_model, etc.).
- database.py — SQLAlchemy engine, SessionLocal, get_db() dependency generator for FastAPI routes.

7.3 Schemas — app/schemas/schemas.py

Pydantic models for JSON bodies and responses (camelCase field names to match typical frontend JSON conventions):

- Import, analyze, playlist create/get, Anki generation request.

7.4 Routes — app/api/v1/routes/

File	Endpoint(s)	Behavior
`lyrics.py`	`POST /import-lyrics`	Fetch or accept raw text → clean → ensure user exists → insert `Song` + `Lyric` → return ids ar
`analysis.py`	`POST /analyze-lyrics`	Load latest lyric for `songId` → delete existing vocab for that song → run NLP → insert `Vocabu
`anki.py`	`POST /generate-anki`	Load selected vocab rows by id → build CSV string → return as downloadable `text/csv`
`playlists.py`	`POST /playlist/create`, `GET /playlist/{id}`	Create playlist; fetch playlist with joined songs and total vocab count across those songs

7.5 Services — app/services/ (modular logic)

Module	Responsibility
`lyrics_importer.py`	`raw` returns string as-is; `url` does GET + BeautifulSoup `get_text`
`lyrics_cleaner.py`	Removes bracketed segments (e.g. `[Chorus]`), trims lines
`nlp_pipeline.py`	spaCy load (cached singleton), POS filter to verb/noun/adjective, dedupe by (word, POS), context line, lemma/infinitive for verbs
`translation_service.py`	Small stub dictionary + fallback string
`anki_exporter.py`	CSV column order fixed for Anki import

7.6 Migrations — alembic/

- alembic.ini — default DB URL (overridden by env.py reading settings.database_url).
- alembic/env.py — wires metadata from SQLAlchemy Base.
- versions/0001_initial.py — creates all MVP tables, indexes, and constraints.

---

8. Request/response flow (end-to-end)

8.1 Import lyrics

Browser posts JSON to POST /api/v1/import-lyrics with sourceType, sourceValue, title, artist, userId.
Backend obtains raw text (paste or HTTP fetch).
Cleaner produces clean_text.
User bootstrap: if userId not in DB, insert placeholder User with that primary key.
Insert Song then Lyric, commit.
Response returns songId, lyricId, cleanedLyrics, detectedLanguage (currently always es in MVP).

8.2 Analyze lyrics

Browser posts { songId } to POST /api/v1/analyze-lyrics.
Backend selects the most recent Lyric for that song.
Deletes existing vocabulary_entries for that song (re-analysis replaces vocab).

NLP extracts tokens → maps to verb/noun/adjective → adds translation + context line.

Persists each row with is_selected=True.

Returns:

- entries: full list with database ids (needed for export selection).
- grouped: same items organized by POS key for UI sections.

8.3 Generate Anki CSV

Browser posts { songId, selectedVocabularyIds: [...] } to POST /api/v1/generate-anki.

Backend loads matching rows; if none, 404.

CSV columns (exact order):

- Spanish Word
- English Translation
- Context Sentence
- Part of Speech
- Infinitive Form (if applicable)

Response headers suggest filename lingroove-anki.csv.

8.4 Playlists

- Create: inserts Playlist for userId (no auto-user bootstrap here in current code — user should exist).
- Get: returns metadata, list of songs in playlist, and count of vocabulary rows for songs linked to that playlist.

Gap: There is not yet a dedicated endpoint in this MVP to add a song to a playlist from the UI; the data model supports it via playlist_songs.

---

9. Frontend structure

9.1 Pages (apps/web/src/app/)

Route	File	Purpose
`/`	`page.tsx`	Landing / marketing hero
`/dashboard`	`dashboard/page.tsx`	Import form entry point
`/analysis/[songId]`	`analysis/[songId]/page.tsx`	Run analysis, show grouped vocab, toggles for export selection
`/playlist/[id]`	`playlist/[id]/page.tsx`	Fetch and display playlist detail
`/export`	`export/page.tsx`	Reads query params (`songId`, `ids`), downloads CSV via API

Next.js note: /export uses useSearchParams wrapped in Suspense to satisfy Next.js static generation constraints.

9.2 Components

- TopNav.tsx — Global navigation.
- ImportLyricsForm.tsx — Client form; calls importLyrics then navigates to analysis route.
- VocabGroupPanel.tsx — Renders list of vocab cards with checkbox selection.

9.3 API client — src/lib/api.ts

- Centralizes fetch URLs using NEXT_PUBLIC_API_BASE_URL.
- Wraps network failures with a clear error if the API is unreachable.
- Parses FastAPI detail when present for failed HTTP responses.

9.4 Styling — globals.css + tailwind.config.ts

- CSS variables for background, surfaces, text, accent green.
- Utility classes .card, .button-primary, .button-secondary.
- Important: Custom Tailwind colors defined as plain CSS variables do not support /opacity modifiers inside @apply the same way as built-in palettes; hover styles use solid border-accent etc.

---

10. Features: what is fully built vs partial

Implemented and wired

- Lyric import (URL + raw).
- Cleaning pipeline (basic).
- NLP extraction with POS categories (verbs, nouns, adjectives).
- Persistence of songs, lyrics, vocabulary.
- Playlist create + playlist detail API + basic playlist page.
- Anki CSV export with required columns.
- Dark responsive UI skeleton for core flows.

Simplified / placeholder

- Translation: stub dictionary + generic fallback strings — not a real translation engine.
- URL lyrics: generic HTML text extraction — may include site chrome; no per-site lyric extractor.
- Auth: no login; hard-coded userId in UI; import route auto-creates user row for FK integrity.
- Language detection: response field exists; logic does not auto-detect language yet.
- Playlist membership: model supports many-to-many; UI/API for “add song to playlist” not fully productized in MVP.

- Highlighted lyrics: analysis page shows context lines and grouped vocab; full inline highlighting in a single lyrics view is minimal.

Future-ready fields (not used yet)

- Song.external_track_id — reserved for Spotify or other music IDs.
- translation_provider setting — reserved for switching providers.
- Service-oriented NLP/translation modules — intended extension points.

---

11. Operations: how to run locally

See README.md for:

- First-time install (venv, pip install, spaCy model, alembic upgrade, npm install).
- Start after installation: Docker Postgres, Uvicorn, npm run dev.

Health check: GET http://localhost:8000/health

---

12. Testing

- apps/api/tests/test_health.py — verifies /health returns ok.
- Frontend: npm run lint in apps/web.

---

13. Known fixes applied during MVP bring-up

- Tailwind @apply + custom color opacity: invalid classes removed; use solid accent borders or refactor colors to RGB CSS variables if opacity variants are needed.
- Next.js useSearchParams: export page wrapped in Suspense.
- Import failing when user missing: import-lyrics ensures User exists for userId.
- Frontend “Failed to fetch”: improved error messaging when API is down or URL misconfigured.

---

14. Summary

Lingroove’s MVP is a monorepo with a Next.js client and FastAPI server backed by PostgreSQL, using spaCy for Spanish POS/lemma-based vocabulary extraction and a CSV exporter formatted for Anki. The codebase separates routes, models, schemas, and services so you can evolve lyric sources, translation, and music integrations without rewriting the core API surface.

Generated as project documentation. For the latest file paths and line-level behavior, refer to the repository source.